

Informe final - Detector de tendencias

Proyecto TPI con cuatro motores de datos: MongoDB, Cassandra, Redis y Neo4j. Este informe resume como se levantan las conexiones, como se crean las estructuras físicas, como se cargan los datos, como se consultan y como se ejecutan las operaciones CRUD disponibles desde el dashboard.

1. Flujo general de ejecucion

El proyecto centraliza la ejecucion en app/main.py. Cada comando usa codigo propio del proyecto y no requiere operar manualmente las bases salvo para verificar durante la exposicion.

```
docker compose up -d
python -m app.main test
python -m app.main setup
python -m app.main load
python -m app.main validate
python -m app.main queries
python -m app.main dashboard
```

Comando	Responsabilidad
test	Verifica conectividad con MongoDB, Cassandra, Redis y Neo4j usando app/connections.py.
setup	Crea colecciones, indices, tablas, constraints, indices y metadata de claves.
load	Genera/carga el dataset comun en los cuatro motores mediante app/loaders/*.py.
validate	Cuenta y valida coherencia minima de los datos cargados.
queries	Ejecuta consultas demostrativas de todos los motores desde app/queries/run_queries.py.
dashboard	Abre la UI Dash con lectura general y pestana Operaciones CRUD.

2. Conexion a bases de datos

Todas las conexiones viven en app/connections.py y toman parametros desde app/config.py, que a su vez lee el archivo .env.

```
# MongoDB
client = MongoClient(MONGO_URI)
db = client[MONGO_DB]

# Redis
Redis(host=REDIS_HOST, port=REDIS_PORT, db=REDIS_DB, decode_responses=True)

# Neo4j
GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

# Cassandra
Cluster([CASSANDRA_HOST], port=CASSANDRA_PORT).connect()
```

3. MongoDB

Conexion

MongoDB se conecta con get_mongo_db() en app/connections.py. La base por defecto es tendencias_db y se configura con MONGO_URI y MONGO_DB.

Estructuras fisicas

app/models/mongo_schema.py crea las colecciones productos, usuarios y categorias. Tambien crea indices unicos y de busqueda para los campos usados por consultas y CRUD.

```
db.productos.create_index("producto_id", unique=True)
db.productos.create_index("categoria_id")
db.productos.create_index("nombre")
db.usuarios.create_index("usuario_id", unique=True)
db.usuarios.create_index("email", unique=True)
db.categorias.create_index("categoria_id", unique=True)
db.categorias.create_index("nombre", unique=True)
```

Carga

app/loaders/mongo_loader.py usa update_one(..., upsert=True) para cargar o actualizar documentos maestros desde el dataset comun.

```
db.productos.update_one(
    {"producto_id": producto["producto_id"]},
    {"$set": producto},
    upsert=True
)
```

Consultas

? query_counts(): conteo de usuarios, productos y categorias.

? query_productos_mayor_precio(): ranking por precio con limite configurable.

? query_stock_bajo(): productos bajo el umbral QUERY_LOW_STOCK_THRESHOLD.

? query_productos_por_categoria(): aggregation con \$group y \$lookup.

? query_categoria_por_id(), query_producto_por_id(), query_usuario_por_id(): busqueda por ID ingresado o tomado desde datos reales para demo.

CRUD expuesto en dashboard

? mongo_upsert_product(): crea o actualiza un producto en productos y verifica con find_one.

? mongo_delete_product(): elimina un producto y verifica que find_one devuelva None.

Guia terminal MongoDB

```
docker exec -it tpi_mongo mongosh tendencias_db
```

```
// Crear o actualizar lo mismo que el dashboard
```

```
db.productos.updateOne(
  { producto_id: "prod_demo_vivo" },
  { $set: {
    producto_id: "prod_demo_vivo",
    nombre: "Producto demo vivo",
    categoria_id: "cat_demo_vivo",
    marca: "Demo",
    precio: 12345.67,
    stock: 9,
    fecha_alta: new Date().toISOString()
  } },
  { upsert: true }
)
```

```
// Verificar persistencia
```

```
db.productos.findOne({ producto_id: "prod_demo_vivo" }, { _id: 0 })
```

```
// Eliminar y verificar
```

```
db.productos.deleteOne({ producto_id: "prod_demo_vivo" })
```

```
db.productos.findOne({ producto_id: "prod_demo_vivo" }, { _id: 0 })
```

4. Cassandra

Conexion

Cassandra se conecta con get_cassandra_session() y luego se selecciona el keyspace CASSANDRA_KEYSPACE.

Estructuras fisicas

app/models/cassandra_schema.py crea el keyspace tendencias y tablas modeladas por consulta. Cassandra no se modela por entidad generica, sino por patron de lectura.

Tabla	Clave primaria / Uso
eventos_por_producto	PRIMARY KEY ((producto_id, fecha), timestamp, evento_id). Eventos de un producto por fecha.
eventos_por_usuario	PRIMARY KEY ((usuario_id, fecha), timestamp, evento_id). Actividad de un usuario por fecha.
eventos_por_categoria	PRIMARY KEY ((categoria_id, fecha), timestamp, evento_id). Eventos por categoria y fecha.
eventos_por_tipo	PRIMARY KEY ((tipo_evento, fecha), timestamp, evento_id). Eventos por comportamiento.

resumen_diario	PRIMARY KEY ((fecha), producto_id). Metricas agregadas por producto y dia.
tendencias_por_categoria_fecha	PRIMARY KEY ((categoria_id, fecha), score_tendencia, producto_id). Top por categoria y fecha.

Carga

app/loaders/cassandra_loader.py inserta cada evento logico en varias tablas orientadas a consulta y luego calcula resumen_diario y tendencias_por_categoria_fecha.

```
session.execute(insert_producto, (...))
session.execute(insert_usuario, (...))
session.execute(insert_categoria, (...))
session.execute(insert_tipo, (...))
session.execute(insert_resumen, (...))
session.execute(insert_tendencia_categoria, (...))
```

Consultas

? query_eventos_por_producto(), query_eventos_por_usuario(), query_eventos_por_categoria(), query_eventos_por_tipo(): usan partition keys reales tomadas desde filas existentes.

? query_resumen_diario(): devuelve resumen para fecha existente o seleccionada desde dashboard.

? query_tendencias_por_categoria_fecha(): top por categoria y fecha.

? query_top_tendencias_resumen_diario(): top diario ordenado por score_tendencia.

CRUD expuesto en dashboard

? cassandra_upsert_daily_summary(): INSERT en resumen_diario. En Cassandra INSERT funciona como upsert para la misma clave.

? cassandra_delete_daily_summary(): DELETE por fecha + producto_id y verificacion con SELECT.

Guia terminal Cassandra

```
docker exec -it tpi_cassandra cqlsh
USE tendencias;

-- Crear o actualizar fila de resumen_diario
INSERT INTO resumen_diario (
    fecha, producto_id, categoria_id, total_eventos,
    total_vistas, total_clicks, total_búsquedas,
    total_favoritos, total_compras, score_tendencia
) VALUES (
    '2026-06-01', 'prod_demo_vivo', 'cat_demo_vivo', 10,
    4, 2, 1, 1, 2, 25.0
);

-- Verificar persistencia
SELECT * FROM resumen_diario
WHERE fecha = '2026-06-01' AND producto_id = 'prod_demo_vivo';

-- Eliminar y verificar
DELETE FROM resumen_diario
WHERE fecha = '2026-06-01' AND producto_id = 'prod_demo_vivo';
SELECT fecha, producto_id FROM resumen_diario
WHERE fecha = '2026-06-01' AND producto_id = 'prod_demo_vivo';
```

5. Redis

Conexion

Redis se conecta con get_redis_client() en app/connections.py usando decode_responses=True para trabajar con strings legibles.

Estructuras fisicas

Redis no requiere schema, pero app/models/redis_keys.py documenta las convenciones de claves y setup_redis() guarda metadata.

Clave	Uso
-------	-----

trending:global	Sorted set global de productos por score.
trending:cat:<categoria_id>	Sorted set por categoria.
cache:top10_global	Cache JSON del top global con TTL.
contador:eventos_total	Contador incremental de eventos procesados.
sesion:<usuario_id>	Hash temporal de sesion de usuario.

Carga

app/loaders/redis_loader.py recorre eventos, incrementa scores con ZINCRBY, actualiza contador, crea cache y sesiones con TTL.

```
redis_client.zincrby(TRENDING_GLOBAL_KEY, score, producto_id)
redis_client.zincrby(f"{TRENDING_CATEGORY_PREFIX}{categoria_id}", score, producto_id)
redis_client.incr(EVENT_COUNTER_KEY)
redis_client.setex(CACHE_TOP10_GLOBAL_KEY, 300, json.dumps(top_global))
redis_client.hset(session_key, mapping={...})
redis_client.expire(session_key, 3600)
```

Consultas

? consulta_ranking_global(): ZREVRANGE sobre trending:global.

? consulta_ranking_por_categoria(): SCAN de trending:cat:* y ZREVRANGE por categoria.

? consulta_cache_top10(): GET de cache:top10_global y TTL.

? consulta_contador_eventos(): GET de contador:eventos_total.

? consulta_sesiones(): SCAN sesion:* y HGETALL por muestra.

CRUD expuesto en dashboard

? redis_upsert_global_score(): ZADD trending:global score producto_id y verifica con ZSCORE.

? redis_delete_global_score(): ZREM trending:global producto_id y verifica que ZSCORE devuelva null.

Guia terminal Redis

```
docker exec -it tpi_redis redis-cli

# Crear o actualizar score global
ZADD trending:global 99 prod_demo_vivo

# Verificar persistencia
ZSCORE trending:global prod_demo_vivo
ZREVRANGE trending:global 0 9 WITHSCORES

# Eliminar y verificar
ZREM trending:global prod_demo_vivo
ZSCORE trending:global prod_demo_vivo
```

6. Neo4j

Conexion

Neo4j se conecta con get_neo4j_driver() usando el protocolo Bolt y credenciales NEO4J_USER / NEO4J_PASSWORD.

Estructuras fisicas

app/models/neo4j_schema.py crea constraints de unicidad e indices. app/loaders/neo4j_loader.py crea nodos y relaciones del grafo.

```
CREATE CONSTRAINT usuario_id_unique IF NOT EXISTS
FOR (u:Usuario) REQUIRE u.usuario_id IS UNIQUE;
CREATE CONSTRAINT producto_id_unique IF NOT EXISTS
FOR (p:Producto) REQUIRE p.producto_id IS UNIQUE;
CREATE CONSTRAINT categoria_id_unique IF NOT EXISTS
FOR (c:Categoria) REQUIRE c.categoria_id IS UNIQUE;
```

Nodos	(:Usuario), (:Producto), (:Categoria)
Catalogo	(:Producto)-[:PERTENECE_A]->(:Categoria)
Interaccion	(:Usuario)-[:VIO CLICK BUSCO FAVORITO COMPRO]->(:Producto) con propiedad cantidad.
Interes	(:Usuario)-[:INTERESADO_EN]->(:Categoria) con propiedad cantidad.

Carga

app/loaders/neo4j_loader.py hace MERGE de usuarios, categorias y productos. Para cada evento incrementa la relacion usuario-producto correspondiente y tambien la relacion INTERESADO_EN hacia la categoria.

```
MERGE (u:Usuario {usuario_id: $usuario_id})
MERGE (p:Producto {producto_id: $producto_id})
MERGE (u)-[r:<TIPO_EVENTO>]->(p)
ON CREATE SET r.cantidad = 1
ON MATCH SET r.cantidad = coalesce(r.cantidad, 0) + 1
```

```
MERGE (u)-[r:INTERESADO_EN]->(c)
ON CREATE SET r.cantidad = 1
ON MATCH SET r.cantidad = coalesce(r.cantidad, 0) + 1
```

Consultas

? query_counts(): cantidad de nodos y eventos representados por relaciones.

? query_usuarios_mas_activos(): usuarios con mas actividad total.

? query_productos_mas_conectados(): ahora filtra por tipo de evento recibido desde dashboard, por ejemplo FAVORITO o COMPRO.

? query_categorias_con_mas_interes(): categorias con mas usuarios y eventos de interes.

? query_usuarios_por_producto_evento(): permite buscar usuarios que realizaron un evento sobre un producto.

? query_recomendaciones_sample(): recomienda productos de categorias de interes que el usuario todavia no interactuo, ponderando popularidad.

CRUD expuesto en dashboard

? neo4j_upsert_user_event(): crea o incrementa una relacion usuario-producto para el tipo de evento elegido.

? neo4j_delete_user_event(): elimina esa relacion y verifica que no queden relaciones del tipo elegido entre usuario y producto.

Guia terminal Neo4j

```
docker exec -it tpi_neo4j cypher-shell -u neo4j -p neo4j123
```

```
// Crear o actualizar evento usuario-producto (usar VIO, CLICK, BUSCO, FAVORITO o COMPRO)
MERGE (u:Usuario {usuario_id: 'usr_demo_vivo'})
MERGE (p:Producto {producto_id: 'prod_demo_vivo'})
MERGE (u)-[r:COMPRO]->(p)
ON CREATE SET r.cantidad = 1, r.fecha_ultima = datetime()
ON MATCH SET r.cantidad = coalesce(r.cantidad, 0) + 1,
            r.fecha_ultima = datetime()
RETURN u.usuario_id, p.producto_id, type(r), r.cantidad;

// Verificar persistencia
MATCH (u:Usuario {usuario_id: 'usr_demo_vivo'})-[r:COMPRO]->(p:Producto {producto_id: 'prod_demo_vivo'})
RETURN u.usuario_id, p.producto_id, type(r), r.cantidad;

// Eliminar y verificar
MATCH (:Usuario {usuario_id: 'usr_demo_vivo'})-[r:COMPRO]->(:Producto {producto_id: 'prod_demo_vivo'})
DELETE r;
MATCH (:Usuario {usuario_id: 'usr_demo_vivo'})-[r:COMPRO]->(:Producto {producto_id: 'prod_demo_vivo'})
RETURN count(r) AS relaciones_restantes;
```

7. Dashboard y operaciones CRUD

La UI esta en app/dashboard/app.py. Tiene una pestana Dashboard para consultas y otra pestana Operaciones CRUD. Cada operacion del dashboard llama a app/crud_operations.py y muestra un JSON con statement, params y verification. La verificacion siempre realiza una lectura posterior contra el motor correspondiente.

```
python -m app.main dashboard
# Abrir http://127.0.0.1:8050
# Pestana: Operaciones CRUD
```

8. Criterio de no hardcodeo

No se usan IDs fijos de demo en consultas operativas. Mongo toma muestras reales desde la base; Cassandra toma filas existentes para resolver partition keys; Redis y Neo4j usan limites configurables; Neo4j acepta tipo de evento como parametro desde el dashboard. Los nombres de tablas, colecciones, labels, relaciones y claves Redis son parte del modelo fisico y por eso permanecen en codigo.

9. Archivos clave

Conexiones	app/connections.py
Configuracion	app/config.py y .env
Setup fisico	app/models/mongo_schema.py, cassandra_schema.py, redis_keys.py, neo4j_schema.py
Carga	app/loaders/mongo_loader.py, cassandra_loader.py, redis_loader.py, neo4j_loader.py
Consultas	app/queries/mongo_queries.py, cassandra_queries.py, redis_queries.py, neo4j_queries.py
CRUD	app/crud_operations.py
Dashboard	app/dashboard/app.py y app/dashboard/dashboard_data.py